

Name: Michael Lartey

Real Estate Agency Portal

CIS 344 Final Project Report

Professor Yanilda Peralta Ramos

Semester: Spring 2026

1. Project Purpose

The Real Estate Agency Portal is a web-based platform developed using PHP and MySQL. The purpose of the project is to simulate a realistic real estate agency environment where three types of users — agents, buyers, and renters — can interact with property listings and agency services.

The platform allows agents to add and manage property listings, while buyers and renters can browse available properties and submit inquiries. The system tracks users, listings, inquiries, favorites, and transactions through a relational MySQL database, and implements secure login functionality with role-based access control.

The project was built on a PHP starter scaffold provided by the professor, with students responsible for connecting the platform to a MySQL database, implementing required PHP methods, and completing a secure login system.

2. Database Design

The database is named `real_estate_portal_db` and contains five tables that model the core data of the portal. The tables are linked through foreign key relationships to enforce data integrity.

2.1 Users Table

Stores all users of the portal. Each user has a unique `userId` (auto-incremented), a unique `userName`, contact information, a hashed password, and a `userType` which is one of three ENUM values: `agent`, `buyer`, or `renter`. The `userId` is the primary key.

2.2 Properties Table

Stores all real estate listings. Each property includes a title, property type, address, city, price, and status (`available`, `sold`, or `rented`). The `agentId` column is a foreign key that references the Users table, linking each property to the agent who listed it.

2.3 Inquiries Table

Stores messages sent by buyers or renters about a specific property. Each inquiry includes a `userId` and `propertyId` as foreign keys, a message, and an `inquiryDate` timestamp recorded automatically using `NOW()`.

2.4 Transactions Table

Records completed sales or rentals. Each transaction references both a property and a user through foreign keys, and stores the transaction type (`sale` or `rental`), date, and amount.

2.5 Favorites Table

Allows buyers and renters to save properties they are interested in. Each record links a user to a property through foreign keys and stores the date the property was saved.

3. Stored Procedures, View, and Trigger

3.1 Stored Procedure: AddOrUpdateUser

This procedure accepts a `userId`, `userName`, `contactInfo`, `passwordHash`, and `userType` as inputs. If the `userId` does not exist in the `Users` table, it inserts a new user record. If the `userId` already exists, it updates the existing user with the new values. This allows the portal to handle both new registrations and profile updates through a single reusable procedure.

3.2 Stored Procedure: ProcessTransaction

This procedure accepts a `propertyId`, `userId`, `transactionType`, and `amount`. It inserts a new record into the `Transactions` table and then updates the status of the corresponding property to either `sold` or `rented` depending on the transaction type. This ensures that the property status is always kept in sync with transaction data.

3.3 View: PropertyListingView

The `PropertyListingView` is a database view that joins the `Properties` and `Users` tables to display a user-friendly listing of properties. It includes the property title, property type, city, price, status, and the agent's username. This view simplifies queries by pre-joining the relevant tables.

3.4 Trigger: AfterTransactionInsert

This trigger fires automatically after every `INSERT` on the `Transactions` table. It checks the transaction type and updates the corresponding property status to `sold` if the transaction is a sale, or `rented` if it is a rental. The trigger acts as a safety net to ensure property statuses are always accurate, even if the update logic in the stored procedure is bypassed.

4. PHP Functionality

The PHP backend is organized using a class-based structure. The core database logic is contained in the `RealEstateDatabase` class, which uses PDO (PHP Data Objects) for all database interactions. The following methods were implemented:

4.1 `addUser()`

Inserts a new user into the `Users` table using a prepared statement with bound parameters. Accepts `userName`, `contactInfo`, `passwordHash`, and `userType` as inputs and returns `true` on success.

4.2 `addProperty()`

Inserts a new property listing into the Properties table. Accepts title, propertyType, address, city, price, status, and agentId as inputs. Only accessible to logged-in agents through role-based access control.

4.3 addInquiry()

Inserts a new inquiry into the Inquiries table. Accepts userId, propertyId, and message as inputs. The inquiryDate is set automatically using the MySQL NOW() function to record the exact timestamp of submission.

4.4 getUserDetails()

Retrieves a user record along with all related inquiries, favorites, and transactions using LEFT JOIN operations across four tables. This provides a complete view of a user's activity on the platform.

4.5 getAllProperties()

Retrieves all property listings joined with agent names from the Users table, ordered by propertyId descending so the most recently added properties appear first.

5. Login and Session Handling

The login system is implemented in login.php and uses PHP sessions to maintain user state across pages. When a user submits the login form, the system looks up the username in the database using getUserByUsername() and verifies the submitted password against the stored hash using PHP's built-in password_verify() function.

If authentication succeeds, the user object is stored in the \$_SESSION superglobal and the user is redirected to their dashboard. If authentication fails, an error message is displayed. Sessions are started at the top of every page through config.php using session_start(), which is called only if a session is not already active.

Password storage is handled securely in register.php using PHP's password_hash() function with the PASSWORD_DEFAULT algorithm, which currently uses bcrypt. Plain text passwords are never stored in the database.

6. Role-Based Access Control

The platform supports three user roles: agent, buyer, and renter. Each role has different access privileges enforced through PHP session checks on individual pages:

- Agents: Can access the Add Property page and manage listings. The navigation bar shows agent-specific options when an agent is logged in.
- Buyers and Renters: Can browse property listings, view property details, and submit inquiries. They are directed to the Browse Properties action from their dashboard.

- **Unauthenticated Users:** Can view property listings but cannot submit inquiries or access the dashboard. The navigation bar shows Login and Register links instead of Dashboard and Logout.

7. Challenges and Solutions

7.1 MySQL Workbench vs XAMPP MySQL

A major challenge encountered was that MySQL Workbench and XAMPP run separate MySQL server instances on different ports. The database was initially created in MySQL Workbench but the PHP application running through XAMPP could not find it. The solution was to recreate the entire database schema in phpMyAdmin, which is XAMPP's built-in MySQL management interface.

7.2 DELIMITER in phpMyAdmin

When importing stored procedures and triggers into phpMyAdmin, the standard DELIMITER \$\$ syntax used in MySQL Workbench caused syntax errors. The solution was to change the delimiter setting in phpMyAdmin's SQL tab from ; to \$\$ before running procedures and triggers.

7.3 Folder Structure and File Paths

The starter code was structured with subfolders (config/, classes/, includes/) but files were initially placed in a flat directory. This caused require_once paths to fail. The solution was to clone the original starter code repository directly into XAMPP's htdocs folder using git clone, which preserved the correct folder structure.

7.4 Duplicate Foreign Key Constraints

The model designer in MySQL Workbench generated duplicate foreign key constraint names when forward engineering the schema, causing errors. The solution was to abandon the visual model designer and write the CREATE TABLE statements directly as SQL scripts, which gave full control over constraint naming.

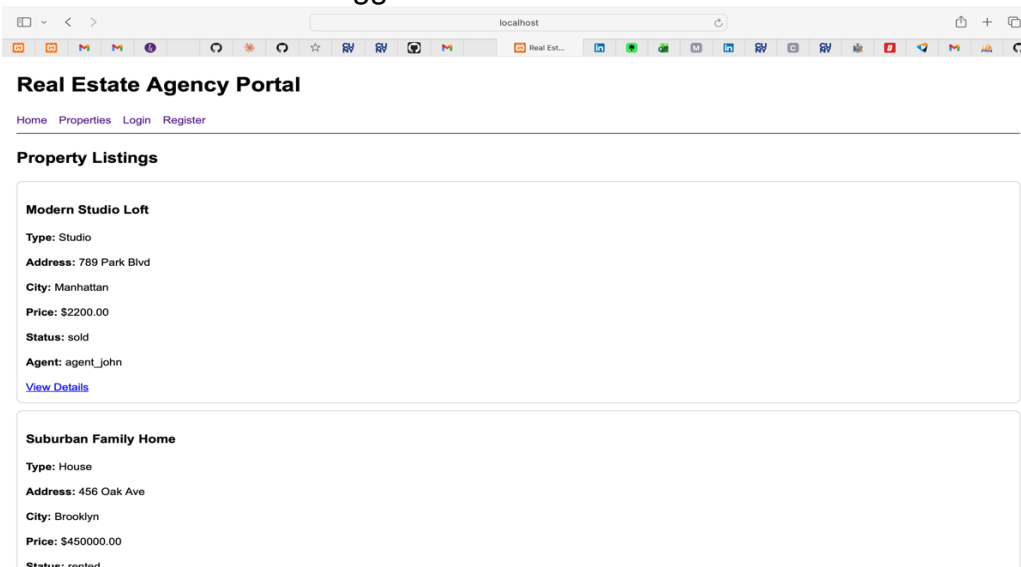
8. Screenshots of the Working Platform

The following screenshots demonstrate the working Real Estate Agency Portal running on a local XAMPP server.

8.1 Property Listings Page

The properties page displays all available listings pulled from the Properties table joined with agent names. Each listing shows the title, type, address, city, price, status, and agent. Properties are correctly showing statuses of sold and rented as updated by the

AfterTransactionInsert trigger.



8.2 Login Page

The login page accepts a username and password. Password verification uses `password_verify()` to compare the submitted password against the bcrypt hash stored in the database. Successful login redirects the user to their dashboard.

Real Estate Agency Portal

[Home](#) [Properties](#) [Login](#) [Register](#)

Login

Username

Password

Login

© Spring 2026 Real Estate Agency Portal

8.3 Register Page

The registration form collects username, contact info, password, and user type. The password is hashed using `password_hash()` before being stored in the database. Users

can register as an agent, buyer, or renter.

Real Estate Agency Portal

[Home](#) [Properties](#) [Login](#) [Register](#)

Register

Username

Contact Info

Password

User Type

Select role 

© Spring 2026 Real Estate Agency Portal

8.4 Property Details Page

Clicking View Details on any property opens the property details page, which shows the full information for that specific listing including its current status as updated by database triggers.

Real Estate Agency Portal

[Home](#) [Properties](#) [Login](#) [Register](#)

Property Details

Modern Studio Loft

Type: Studio

Address: 789 Park Blvd

City: Manhattan

Price: \$2200.00

Status: sold

Agent: agent_john

© Spring 2026 Real Estate Agency Portal

8.5 Add Property Page

The Add Property page is accessible to logged-in agents and provides a form to enter all property details including title, type, address, city, price, and initial status. Submitted

data is inserted into the Properties table using the `addProperty()` method.

Real Estate Agency Portal

[Home](#) [Properties](#) [Dashboard](#) [Logout](#)

Add Property

Title

Property Type

Address

City

Price

Status

© Spring 2026 Real Estate Agency Portal

8.6 Submit Inquiry Page

The inquiry submission form allows logged-in buyers or renters to send a message about a property. The inquiry is saved to the Inquiries table using the `addInquiry()` method with the current timestamp recorded automatically.

Real Estate Agency Portal

[Home](#) [Properties](#) [Dashboard](#) [Logout](#)

Submit Inquiry

Message

© Spring 2026 Real Estate Agency Portal

8.7 Dashboard Page

The dashboard page greets the logged-in user by name and displays their role. Role-based actions are shown based on user type. A buyer's dashboard shows a Browse

Properties link, while an agent's dashboard would show property management options.

Real Estate Agency Portal

[Home](#) [Properties](#) [Dashboard](#) [Logout](#)

Dashboard

Welcome: mike

Role: buyer

Common Actions

[Browse Properties](#)

© Spring 2026 Real Estate Agency Portal

8.8 phpMyAdmin — Database Tables

The phpMyAdmin interface shows all six objects in `real_estate_portal_db`: the five tables (Favorites, Inquiries, Properties, Transactions, Users) and the PropertyListingView. Each table contains at least three rows of sample data.

The screenshot shows the phpMyAdmin interface for a database named 'real_estate_portal_db' on a localhost server. The top navigation bar includes tabs for Structure, SQL, Search, Query, Export, Import, and Operations. Below this is a 'Filters' section with a search box. The main area displays a table of database structures. Each row represents a table with columns for 'Table', 'Action' (Browse, Structure, Search, Insert, Empty, Drop), 'Rows', and 'Type'. The tables listed are Favorites, Inquiries, Properties, propertylistingview, Transactions, and Users. Below the table list, there is a summary row showing '6 tables' and a total of '~17' rows. At the bottom, there is a 'Create new table' section with input fields for 'Table name' and 'Number of columns' (set to 4), and a 'Create' button. A 'Console' tab is visible at the bottom left.

Table	Action	Rows	Type
☐ Favorites	★ Browse Structure Search Insert Empty Drop	3	InnoDB
☐ Inquiries	★ Browse Structure Search Insert Empty Drop	3	InnoDB
☐ Properties	★ Browse Structure Search Insert Empty Drop	3	InnoDB
☐ propertylistingview	★ Browse Structure Search Insert Edit Drop	~0	View
☐ Transactions	★ Browse Structure Search Insert Empty Drop	3	InnoDB
☐ Users	★ Browse Structure Search Insert Empty Drop	5	InnoDB
6 tables	Sum	~17	InnoDB

↑ ☐ Check all With selected:

Print Data dictionary

Create new table

Table name: Number of columns:

Console

8.9 phpMyAdmin — Stored Procedures

The Routines section of phpMyAdmin shows both stored procedures: AddOrUpdateUser and ProcessTransaction, both of type PROCEDURE.

Routines

☐ Check all

 Export

 Drop



Create new routine

	Name	Type	Returns	
☐	AddOrUpdateUser	PROCEDURE		 Edit  Execute  Export  Drop
☐	ProcessTransaction	PROCEDURE		 Edit  Execute  Export  Drop

8.10 phpMyAdmin — Trigger

The Triggers section shows the AfterTransactionInsert trigger, which fires AFTER INSERT on the Transactions table and automatically updates the corresponding property status.

Triggers

☐ Check all

 Export

 Drop

	Name	Time	Event	
☐	AfterTransactionInsert	AFTER	INSERT	 Edit  Export  Drop

9. Conclusion

The Real Estate Agency Portal successfully demonstrates a full-stack web application built with PHP and MySQL. The platform implements all required features including a relational database with five tables, stored procedures for user management and transaction processing, a view for property listings, and a trigger for automatic status updates.

The PHP implementation includes secure password hashing, session management, role-based access control, and full CRUD operations for users, properties, and inquiries. The project provided practical experience with connecting a PHP application to a MySQL database, working with prepared statements, and building a realistic multi-user web platform.